

agentic-ai-azure-week06-07-multi-agent

Weeks 6-7 - Multi-Agent Orchestration

Agentic AI on Azure - Enterprise Master Class | Handout (slides + notes)

Notes

Welcome. This deck covers 'agentic-ai-azure-week06-07-multi-agent' - Weeks 6-7 - Multi-Agent Orchestration. Frame the problem and why it matters, then walk the class through the learning goal, the enterprise use case, the architecture/flow, the key concepts, a live run in VS Code, and the architect's takeaways. The repo runs locally with no Azure, so demo it live.

Slide 2 - Learning goal

- Implement sequential, concurrent & handoff patterns
- Add human-in-the-loop approval
- Track long-running runs with status polling

Notes

[Learning goal] Walk through these talking points:

- Implement sequential, concurrent & handoff patterns
- Add human-in-the-loop approval
- Track long-running runs with status polling

Ask the class: which of these ideas is new to you?

Slide 3 - Enterprise use case

- Loan Underwriting Pipeline (banking)
- Intake -> (Credit-Risk || Compliance) -> Underwriter
- Edge cases pause for human approval

Notes

[Enterprise use case] Walk through these talking points:

- Loan Underwriting Pipeline (banking)
- Intake -> (Credit-Risk || Compliance) -> Underwriter
- Edge cases pause for human approval

Tip: relate this to a regulated scenario your students know.

Slide 4 - Architecture / flow

- Intake first, then concurrent agents (asyncio.gather)
- Underwriter handoff applies decision rules
- awaiting_approval -> resume via approve endpoint
- Run store keeps state (Cosmos in prod)

Notes

[Architecture / flow] Walk through these talking points:

- Intake first, then concurrent agents (asyncio.gather)
- Underwriter handoff applies decision rules
- awaiting_approval -> resume via approve endpoint
- Run store keeps state (Cosmos in prod)

Demo: trace a single request through these steps live.

Slide 5 - Key concepts

- Concurrency vs. sequential dependencies
- Human-in-the-loop checkpoints + durable state
- A2A + event-driven (Service Bus / Event Grid)

Notes

[Key concepts] Walk through these talking points:

- Concurrency vs. sequential dependencies
- Human-in-the-loop checkpoints + durable state
- A2A + event-driven (Service Bus / Event Grid)

Open the named file in the repo while you explain each point.

Slide 6 - Run it (VS Code - no Azure needed)

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/underwrite
- Runs in MOCK mode - no Azure/keys needed

Notes

[Run it (VS Code - no Azure needed)] Walk through these talking points:

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/underwrite
- Runs in MOCK mode - no Azure/keys needed

Pause for questions before moving on.

Slide 7 - Architect's takeaways

- Pick the pattern that fits the dependencies
- Durable state for long-running approvals
- Handle partial results, compensation, retries

Notes

[Architect's takeaways] Walk through these talking points:

- Pick the pattern that fits the dependencies
- Durable state for long-running approvals
- Handle partial results, compensation, retries

Discussion: when would you NOT choose this approach?